GARCH Audit

Yosri BEN HALIMA, Mathieu CHABIRAND

Volatility asymmetry of the S&P 500, weekly returns

We audit the constant-mean GARCH(1,1) written by the AI on weekly S&P 500 log-returns in percent and compare it with the Python `arch` package. We then study volatility asymmetry with the GJR-GARCH and the news impact curves. *Labels*: [c] computed (repository code, sheet `weekly_spx_ust10`), [a] assumed; no number is simulated or taken from an AI answer. *Slide n*: course deck (NLNPE); *handout p. n*: tutorial handout.

1. Audit

Table 1: the six checkpoints (handout p. 11), on the AI's initial code

Checkpoint	Verdict	Reference	Verification
Residuals	Correct	slide 34	The recursion uses $e_t = r_t - \mu$, not r_t.
Initial variance	Correct	slide 36	Stated: sample variance, divisor T [a], $h_1 = 5.2774$ [c]. The course alternative is $\omega/(1-\alpha-\beta)$.
Likelihood	Correct	slide 39	Minimises $-\sum_t \log \ell_t$ (Gaussian): correct sign.
Constraints	Correct	slide 36	Exponential/softmax reparametrisation: $\omega > 0$, $\alpha, \beta > 0$, $\alpha + \beta < 1$, no clipping. Exact zeros are excluded.
Standard errors	Correct	slides 40-41	Numerical Hessian: a valid estimator of $I(\theta)$. Recomputed at the optimum, it gives plausible t (7.3 to 28.4 [c]) and matches `arch`'s classic covariance to 10^{-4} [c]. BHHH (slide 46) kept as a cross-check.
Scale	Correct	slide 56	The code expects returns in percent, as the prompt asks; we feed it $r_t = 100 \log(P_t/P_{t-1})$, hence $\hat{\omega} = 0.32$ [c].

The rest of the report uses the final version (BHHH standard errors, Hessian as a cross-check).

Table 2: GARCH(1,1) estimates [c], our implementation and the `arch` package

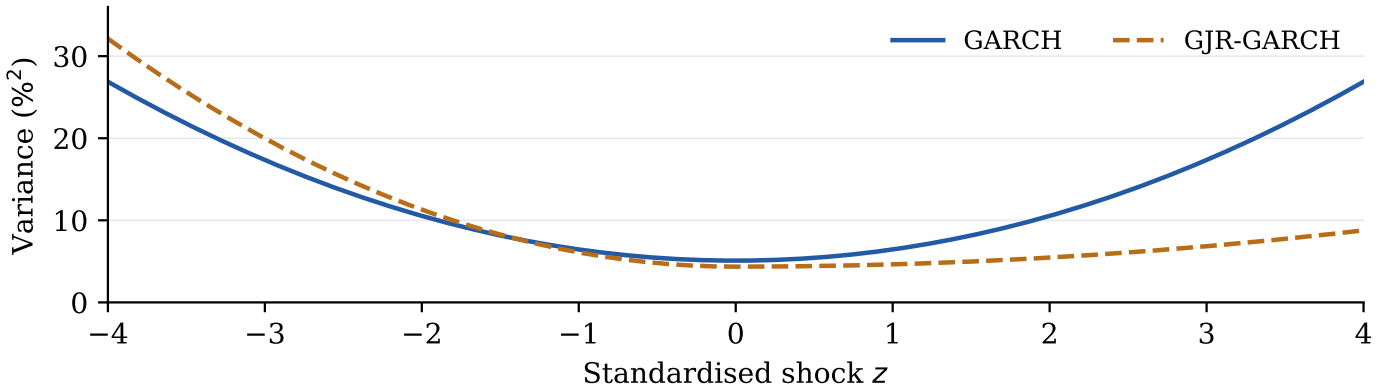
	Our implementation	BHHH s.e.	`arch` GARCH	Absolute gap	Relative gap (%)
μ	0.260789	0.035543	0.260804	0.000015	0.01
ω	0.320176	0.048116	0.320190	0.000014	0.00
α	0.211234	0.015736	0.211268	0.000034	0.02
β	0.739120	0.018465	0.739098	0.000022	0.00

The coefficients agree to two decimals. The log-likelihoods are -5084.787 [c] and -5084.785 [c]. For ω, the BHHH (0.048 [c]), Hessian (0.063 [c]) and robust (0.085 [c]) standard errors diverge: fat tails (excess kurtosis 6.4 [c]) call for caution with Gaussian inference. ARCH-LM rejects a constant variance (12 lags [a], $LM = 374.9$ [c], $p = 7.6 \times 10^{-73}$ [c]), then finds no ARCH effect left after filtering ($p = 0.712$ [c]).

2. Asymmetry

GJR estimated with `arch` (slide 52), robust standard errors: $\hat{\mu} = 0.172$ [c], $\hat{\omega} = 0.368$ [c], $\hat{\alpha} = 0.052$ [c], $\hat{\beta} = 0.744$ [c] and $\hat{\gamma} = 0.272355$ [c] (s.e. 0.056559 [c], $t = 4.815$ [c], $p = 1.47 \times 10^{-6}$ [c], asymptotic). The correlation between this week's return and the absolute value of next week's return is -0.2243 [c] (2,386 pairs): falls tend to precede larger moves, without establishing causality.

We hold the current variance at its unconditional level (handout p. 15), the $\gamma/2$ assuming symmetric innovations [a]: $\bar{h}_G = \omega/(1-\alpha-\beta) = 6.449$ [c] and $\bar{h}_J = \omega/(1-\alpha-\beta-\gamma/2) = 5.360$ [c] ($\%^2$).



Checks of handout p. 15: (1) ✓ curves drawn at $\bar{h}$, not at the last h_t; (2) ✓ indicator on $z < 0$ and $\gamma/2$ in $\bar{h}_J$; (3) × the curves do not coincide for $z > 0$, yet the asymmetry is not on the wrong side: the GARCH spreads over both sides a response the GJR keeps for negative shocks ($\hat{\alpha}_G = 0.211$ vs $\hat{\alpha}_J = 0.052$). Next-week variance at $z = +2$: 10.5 [c] (GARCH) vs 5.5 [c] (GJR); at $z = -2$: 10.5 [c] vs 11.3 [c].

Conclusion. The AI got one point wrong that the course settles: pressed on its numerical-Hessian standard errors, it called them wrong, whereas the Hessian estimates the information matrix just as BHHH does (slides 40-41 and 46).

Appendix A: annotated prompt log

Tags (rule 2 of the handout): **correct**, **wrong** or **unverifiable**, each with the check that settles it.
Checks are reproduced by `main.py` and `tests/` unless stated otherwise. Slide numbers refer to the course deck (NLNPE), TD pages to the handout.

Block A — GARCH implementation, audit and benchmark

User prompt / request	AI response	Tag and check
"Write a Python function that estimates a GARCH(1,1) with a constant mean on weekly returns in percent, by maximum likelihood, without the arch package. State explicitly: how the first conditional variance is initialised, how positivity and stationarity are enforced, how standard errors are computed. Return estimates, standard errors and the maximised log-likelihood."	Gaussian ML implementation with demeaned residuals, initial variance fixed at the sample variance, exponential/softmax reparametrisation of the constraints, standard errors from the inverse numerical Hessian.	**Correct**, standard errors included: the observed Hessian estimates the information matrix (slides 40-41), and checkpoint 5 only rejects a Hessian at the wrong scale or from a non-converged minimiser (TD pp. 11-12). Recomputed at the optimum (`hessian_standard_errors`): positive definite, t from 7.3 to 28.4, equal to `arch`'s classic SE to 1e-4.
"does the implementation check these checkpoints ?"	Checkpoints 1–4 and 6 satisfied under the stated assumptions; checkpoint 5 requires a correction. Softmax constraints exclude exact zero values of alpha and beta.	**Correct** for checkpoints 1-4 and 6 (code read against slides 34, 36, 39). **Wrong** on checkpoint 5: no correction was required (see row 1). The softmax remark is correct.
"adjust to match checkpoint 5"	Replaced the covariance calculation with BHHH: inverse of the sum of observation-score outer products, including the constant mean and recursive variance derivatives, then the delta method. Verified against numerical scores.	**Correct**: BHHH as on slide 46; analytic scores match numerical scores (`test_bhhh_for_both_initializations_matches_numerical_scores`).
"is it long run var or sample var"	Initialization uses the fixed sample variance, divisor T. It is distinct from the fitted long-run variance `omega / (1 - alpha - beta)`. Earlier quoted values belonged to earlier samples.	**Correct**: $h_1 = 5.2774$ = sample variance with divisor T (`results.json`, `custom.initial_variance`). « Earlier quoted values »: **unverifiable** (earlier samples not kept).
"make both possible, up to user's selection"	Added sample and long-run initialization choices. Long-run initialization is recalculated during optimization and its derivatives enter BHHH.	**Correct**: `main.py --initialization long-run` runs; its BHHH scores pass the same numerical-score test.
"add student tests 5% confidence (add a * for significant coefs)"	Added two-sided Wald statistics with a Student-t reference and T−4 degrees of freedom, presented as an approximation: these are not exact finite-sample Student tests.	**Correct** as qualified: ML t-statistics are asymptotically normal; with T = 2,387 the Student-t and normal critical values coincide to 2 decimals.
"by "arch estimate" you mean the garch implementation of arch module ? also add relative difference column in %"	The benchmark uses the Python `arch` package's Gaussian constant-mean GARCH(1,1). Added absolute relative differences, using the absolute package estimate as denominator and unrounded values for calculation.	**Correct**: `arch_model(..., vol='GARCH', p=1, q=1)` as in TD p. 9; all four estimates agree to the second decimal (`benchmark_pass = True`).
"why is Standard errors a "pass" when in first implementation it failed to use BHHH"	Revised its audit: the initial standard-error choice is now called wrong, and the BHHH correction documented.	**Wrong**: its first answer (a « pass ») was right. A numerical Hessian is one of the two routes of checkpoint 5 (row 1).
Summary: explain the initialization difference between our estimator and `arch`, try a sample-variance backcast, then update the report.	`arch` uses a weighted backcast by default. With the sample variance as backcast, presample terms are initialised and the first fitted variance is `omega + (alpha + beta) * backcast`, subject to variance bounds, so it still differs from fixing our first variance to the sample variance.	**Correct**: $0.32019 + (0.21127 + 0.73910) \times 5.27739 = 5.3356$ = `arch.initial_variance` in `results.json`.

Block B — asymmetry and news-impact curves

Prompt / request	AI response	Tag and check
Assignment prompt, summary (slide 14): derive and plot GARCH/GJR news-impact curves for standardized shocks from −4 to 4, evaluated at unconditional variance, and explain the fixed state.	For each fitted model, fix the current variance at its own unconditional variance. GARCH uses `h_bar = omega / (1 - alpha - beta)` and `NIC(z) = omega + (alpha*z² + beta)*h_bar`. GJR under symmetric Gaussian innovations uses `h_bar = omega / (1 - alpha - beta - gamma/2)` and `NIC(z) = omega + (alpha + gamma*1(z<0))*z²*h_bar + beta*h_bar`. The extra GJR term vanishes for positive shocks, but separately fitted curves need not coincide there.	**Correct**: both formulas and both $\bar{h}$ match TD p. 15 (`test_nic_uses_unconditional_state_and_negative_indicator`). Non-coincidence for $z > 0$ is correct: $\hat{\alpha}_{GARCH} = 0.211$ vs $\hat{\alpha}_{GJR} = 0.052$, next variance at $z = +2$ 10.5 vs 5.5.

Appendix B: output of `main.py`

Every number labelled *computed* in the report is produced by this script (and `report/results.json`).
Data: `data/NLNPE_TD_data.xlsx`, sheet `weekly_spx_ust10`.

```
Tutorial 1 A/B: 2387 weekly returns; arch benchmark matches = True
GJR gamma = 0.272355; robust Wald = 4.815
mu: Hessian SE = 0.03555, t = 7.34; arch classic SE = 0.03555
omega: Hessian SE = 0.06278, t = 5.10; arch classic SE = 0.06277
alpha: Hessian SE = 0.02312, t = 9.14; arch classic SE = 0.02313
beta: Hessian SE = 0.02606, t = 28.36; arch classic SE = 0.02606
Numerical results: .\report\results.json
```

Appendix C: code excerpts

The audited estimator and the ARCH-LM test (Block A), the news impact curve (Block B), with their line numbers in the repository.

`src/garch_mle.py : fit_garch11`

```
46  def fit_garch11(returns_pct, initialization="sample"):
47      """Return estimates, BHHH standard errors, log likelihood and fitted variances.
48
49      Input must already contain returns in percent (e.g. 1.2 means 1.2%).
50      The same estimator applies to daily or weekly return observations.
51      initialization="sample": fixed sample variance, with divisor n.
52      initialization="long-run": omega / (1 - alpha - beta), recomputed at
53      every likelihood evaluation, including its derivatives in BHHH scores.
54      Standard errors assume a correctly specified conditional Gaussian model.
55      Their covariance is the inverse sum of observation-score outer products,
56      including the score for the estimated constant mean.
57      """
58      if initialization not in {"sample", "long-run"}:
59          raise ValueError("initialization must be 'sample' or 'long-run'.")
60      y = np.asarray(returns_pct, dtype=float)
61      if y.ndim != 1 or y.size < 10 or not np.isfinite(y).all():
62          raise ValueError("Provide at least 10 finite, one-dimensional returns.")
63      v = float(np.var(y, ddof=0))
64      if v <= 0 or not np.isfinite(v):
65          raise ValueError("Returns must have a positive, finite sample variance.")
66      s = np.sqrt(v)
67
68      def unpack(z):
69          # The third softmax weight is 1 - alpha - beta.
70          a, b, slack = softmax([z[2], z[3], 0.0])
71          with np.errstate(over="ignore", under="ignore"):
72              w = v * np.exp(z[1])
73          return np.array([s * z[0], w, a, b])
74
75      def nll(z):
76          mu, w, a, b = unpack(z)
77          if not np.isfinite([mu, w, a, b]).all() or w <= 0 or a + b >= 1:
78              return np.inf
79          e = y - mu
80          h = np.empty(y.size)
81          h[0] = v if initialization == "sample" else w / (1 - a - b)
82          with np.errstate(over="ignore", divide="ignore", invalid="ignore"):
83              for t in range(1, y.size):
84                  h[t] = w + a * e[t - 1] ** 2 + b * h[t - 1]
85              value = 0.5 * np.sum(np.log(2 * np.pi) + np.log(h) + e**2 / h)
86          return float(value) if np.isfinite(value) else np.inf
87
88      # Multiple starts reduce sensitivity to local optima.
89      fits = []
90      for a, b in [(0.05, 0.90), (0.10, 0.70), (0.15, 0.40)]:
91          slack = 1 - a - b
92          start = [np.mean(y) / s, np.log(slack), np.log(a / slack), np.log(b / slack)]
93          fit = minimize(
94              nll,
95              start,
96              method="L-BFGS-B",
```

```
            options={"maxiter": 2000, "ftol": 1e-12, "gtol": 1e-6},
        )
        if fit.success and np.isfinite(fit.fun):
            fits.append(fit)
    if not fits:
        raise RuntimeError("GARCH optimization failed to converge.")
    fit = min(fits, key=lambda result: result.fun)
    z = fit.x

    mu, w, a, b = unpack(z)
    e = y - mu
    h = np.empty(y.size)
    h[0] = v if initialization == "sample" else w / (1 - a - b)
    # dh[t] = derivative of h[t] with respect to (mu, omega, alpha, beta).
    dh = np.zeros((y.size, 4))
    # Sample initialization is fixed; long-run initialization is parameter-dependent.
    if initialization == "long-run":
        slack = 1 - a - b
        dh[0] = [0, 1 / slack, w / slack**2, w / slack**2]
    for t in range(1, y.size):
        h[t] = w + a * e[t - 1] ** 2 + b * h[t - 1]
        dh[t] = (
            np.array([-2 * a * e[t - 1], 1, e[t - 1] ** 2, h[t - 1]]) + b * dh[t - 1]
        )
    scores = 0.5 * ((e**2 / h - 1) / h)[:, None] * dh
    scores[:, 0] += e / h # Direct derivative of the mean in e[t].

    J = np.array(
        [
            [s, 0, 0, 0],
            [0, w, 0, 0],
            [0, 0, a * (1 - a), -a * b],
            [0, 0, -a * b, b * (1 - b)],
        ]
    )
    # BHHH in transformed coordinates improves numerical scaling.
    # B is a SUM, so its inverse already includes the sample-size factor.
    scores_z = scores @ J
    B = scores_z.T @ scores_z
    if np.isfinite(B).all() and np.linalg.matrix_rank(B) == 4:
        covariance = J @ np.linalg.solve(B, J.T)
        se = np.sqrt(np.diag(covariance))
    else:
        warnings.warn(
            "BHHH information is singular or nonfinite; standard errors unavailable.",
            RuntimeWarning,
            stacklevel=2,
        )
        se = np.full(4, np.nan)

    names = ["mu", "omega", "alpha", "beta"]
    hessian_se = hessian_standard_errors(y, [mu, w, a, b], v, initialization)
    return {
        "hessian_standard_errors": dict(zip(names, hessian_se)),
        "estimates": dict(zip(names, [mu, w, a, b])),
        "standard_errors": dict(zip(names, se)),
        "loglik": -float(fit.fun),
        "initialization": initialization,
        "initial_variance": float(h[0]),
        "conditional_variance": h.copy(),
    }
```

src/garch_mle.py : hessian_standard_errors

```
def hessian_standard_errors(y, theta, sample_variance, initialization="sample", rel_step=1e-4):
    """Standard errors from the inverse numerical Hessian of minus the log likelihood.

    This is the AI's initial covariance choice, kept as a cross-check of BHHH.
    Central differences in natural parameters (mu, omega, alpha, beta), with
    steps proportional to each parameter.
    """
    theta = np.asarray(theta, dtype=float)

    def nll(p):
        mu, w, a, b = p
        e = y - mu
        h = np.empty(y.size)
        h[0] = sample_variance if initialization == "sample" else w / (1 - a - b)
        for t in range(1, y.size):
            h[t] = w + a * e[t - 1] ** 2 + b * h[t - 1]
        return 0.5 * np.sum(np.log(2 * np.pi) + np.log(h) + e**2 / h)

    k = theta.size
    step = rel_step * np.maximum(np.abs(theta), 1e-2)
    H = np.empty((k, k))
    for i in range(k):
        for j in range(k):
            di = np.zeros(k)
            dj = np.zeros(k)
            di[i], dj[j] = step[i], step[j]
            H[i, j] = (
                nll(theta + di + dj) - nll(theta + di - dj)
                - nll(theta - di + dj) + nll(theta - di - dj)
            ) / (4 * step[i] * step[j])
    if not np.isfinite(H).all() or np.linalg.eigvalsh(H).min() <= 0:
        warnings.warn("Numerical Hessian is not positive definite.", RuntimeWarning, stacklevel=2)
        return np.full(k, np.nan)
    return np.sqrt(np.diag(np.linalg.inv(H)))
```

src/tutorial1.py : arch_lm

```python
def arch_lm(residuals, lags=12):
    """ARCH-LM: residual squares regressed on an intercept and lagged squares.

    Pass residuals, NOT their squares: het_arch squares them internally.
    The course convention uses n = T - lags and LM = n R-squared (ddof=0).
    """
    e = np.asarray(residuals, dtype=float)
    if e.ndim != 1 or not np.isfinite(e).all() or len(e) <= 2 * lags:
        raise ValueError("ARCH-LM needs finite residuals and enough observations.")
    lm, pvalue, f, fp = het_arch(e, nlags=lags, ddof=0, result_object=False)
    return {
        "lm": float(lm),
        "pvalue": float(pvalue),
        "f": float(f),
        "f_pvalue": float(fp),
        "lags": lags,
        "auxiliary_n": len(e) - lags,
    }
```

src/tutorial1.py : news_impact_curve

```python
def news_impact_curve(parameters, shocks, gjr=False):
    """Next variance at each model's own unconditional state; shocks are standardised.

    Gaussian innovation symmetry implies E[z^2 1(z<0)] = 1/2 for GJR.
    This state is unrelated to the variance used to initialise the fitted filter.
    """
    w, a, b = (parameters[name] for name in ("omega", "alpha", "beta"))
    gamma = parameters.get("gamma", 0.0) if gjr else 0.0
    persistence = a + b + gamma / 2
    if w <= 0 or a < 0 or a + gamma < 0 or b < 0 or persistence >= 1:
        raise ValueError(
            "A positive finite unconditional variance requires stationary parameters."
        )
    hbar = w / (1 - persistence)
    z = np.asarray(shocks, dtype=float)
    curve = w + (a + gamma * (z < 0)) * z**2 * hbar + b * hbar
    return curve, float(hbar)
```